

Introduction à Linux

Prof : Rachida AIT ABDEOUAHID

Plan du cours

1. Concepts de base
2. Le système d'exploitation Linux
3. Outils Linux
4. Tableur

Concepts de base

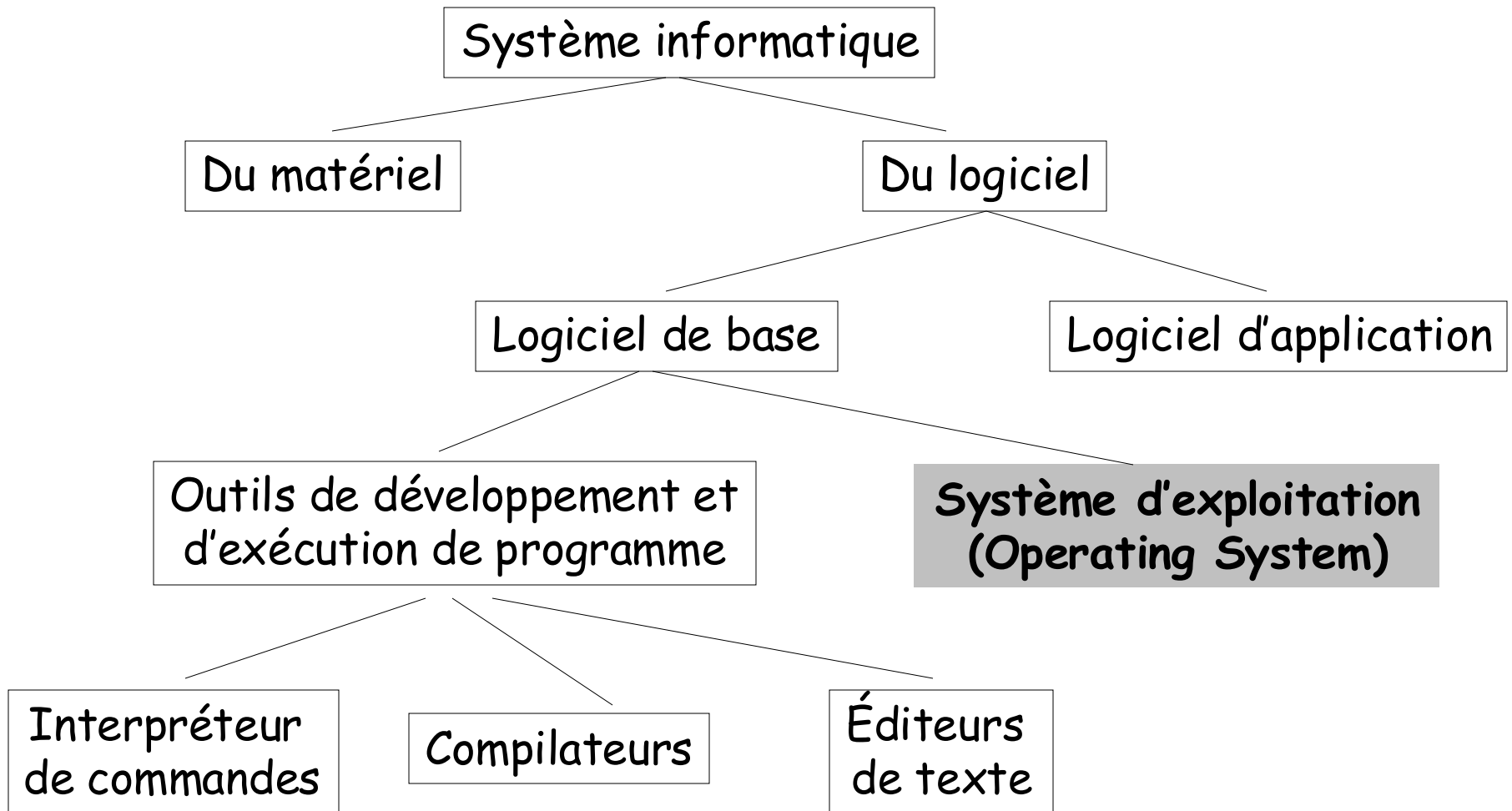
Qu'est ce que l'informatique ?

- L'**informatique** est le traitement automatique de l'information
- L'**information** manipulée pour :
 - l'université : les étudiants, leurs notes, date de leurs inscriptions, leurs emploi du temps ...
 - une entreprise : les produits, temps de production, prix, les stocks ...
- Le **traitement automatique** implique un traitement qui suit des règles qui peuvent être identifiées et éventuellement programmées dans un ordinateur

Qu'est ce qu'un système informatique ?

- Un **système informatique** est un ensemble cohérent de matériels et de logiciels destinés à assurer le traitement automatique d'informations
- Un tel système est composé de 4 entités :
 - le **matériel** (mémoire, processeur, disque, clavier, etc.)
 - le **système d'exploitation** (Windows, Unix, Linux, ...)
 - les **programmes d'applications** (Programmes, Jeux, Réservation d'avion, ...)
 - les **utilisateurs**

Qu'est ce qu'un système informatique ?



Outils de développement et d'exécution de programme

- **L'interpréteur de commandes (shell)** : permet d'accéder aux fonctions du système à l'aide d'un *langage de commande*
- **Les compilateurs** : sont chargés de traduire des programmes écrits dans des langages de haut niveau en une suite d'instructions en *langage machine*
- **Les éditeurs de textes** : permettent de saisir et modifier du texte (par exemple des programmes)

Important :

- Ces outils ne font pas partie du système d'exploitation
- Les compilateurs et éditeurs fonctionnent en mode utilisateur, ils peuvent être changés

Qu'est ce qu'un système d'exploitation ?

- Un **système d'exploitation** est une ensemble de procédures manuelles et automatiques qui permet à un groupe d'utilisateurs de partager efficacement un ordinateur
- Un **système d'exploitation** est un ensemble de procédures cohérentes qui a pour but de gérer la pénurie de ressources
- Un **système d'exploitation** est un ensemble de programmes et de fonctions conçus pour faciliter et optimiser l'utilisation des unités physiques de l'ordinateur
- Le seul programme qui tourne constamment dans une machine
- Il existe plusieurs systèmes d'exploitation. Ils varient selon :
 - le type de matériel
 - la complexité des tâches à effectuer
 - les logiciels qu'ils doivent supporter

Le rôle d'un système d'exploitation

- Fournir à l'utilisateur l'équivalent d'une machine étendue ou virtuelle plus simple à programmer que la machine réelle
- Gérer de manière équitable et optimale l'allocation des processeurs, de la mémoire et des périphériques aux différents programmes qui les sollicitent
- Exemples de systèmes d'exploitation :
 - Mac-OS (le système Macintosh)
 - Windows (NT, 95, 98)
 - Unix, Linux
 - etc.

Le système d'exploitation Linux

C'est quoi Unix ?

- Unix est né au début des années 70 dans les laboratoires Bell
- Unix est un système :
 - **multi-utilisateurs** : plusieurs personnes peuvent partager les ressources de la même machine
 - **multi-tâches** : plusieurs programmes ou logiciels peuvent s'exécuter concurremment
- Il existe plusieurs versions commerciales :
 - AIX de IBM
 - Sun Solaris de SUN Microsystems
 - HP-UX de Hewlett Packard
 - Tru64 Unix de Compaq
 - etc.
- Plusieurs versions d'UNIX sont nées pour PC :
 - Linux
 - FreeBSD
 - OpenBSD
 - NetBSD ...

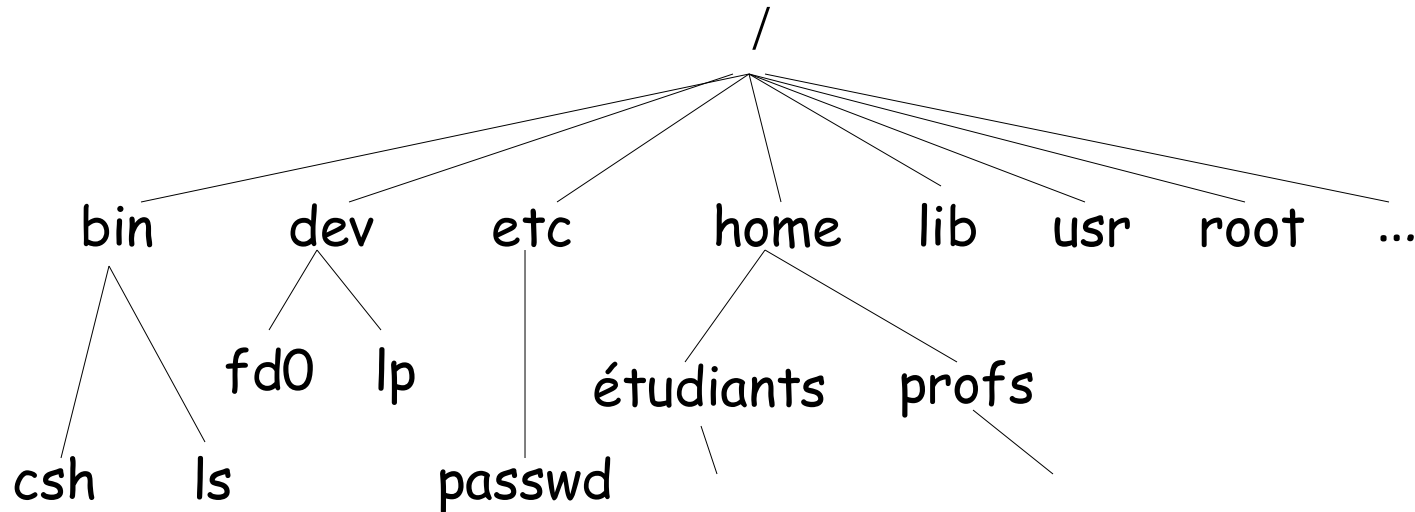
Organisation du système Unix

Le système Unix est organisé en couches :

- **Noyau** : la couche de plus haut niveau, elle assure la communication avec le matériel. Le noyau s'occupe de :
 - la gestion de la mémoire,
 - l'accès aux périphériques (disque dur, lecteur de CD-Rom, clavier, souris, ...),
 - la gestion du réseau, ...
- **Shell** : interprète les ordres de l'utilisateur et les fait exécuter par le noyau. Les ordres peuvent être passés soit directement au clavier, soit en utilisant des outils graphiques de plus haut niveau
- **Applications** : interagissent avec l'utilisateur ou avec d'autres applications et communiquent avec le shell ou avec le noyau

La hiérarchie des répertoires

- Linux définit un système de fichiers hiérarchique avec un certain nombre de répertoires standards



- `/root` est le répertoire d'accueil du super-utilisateur (administrateur)
- `/bin` contient généralement les programmes utiles au démarrage
- `/etc` contient les fichiers de configurations
- `/dev` contient les fichiers relatifs aux devices (périphériques)
- `/home` contient les répertoires des utilisateurs
- `/lib` contient les bibliothèques du système
- `/usr` contient les programmes ajoutés au système

Se logger

- Linux possède un mécanisme d'identification connu sous le nom de **login**
- Pour utiliser un système Linux sur une machine, il faut avoir un **compte** sur cette machine
- Pour se connecter sur une machine il faut rentrer au clavier :
 - son nom d'utilisateur : **login**
 - son mot de passe : **password**
- Le système vérifie la correspondance entre le login et le mot de passe
 - si échec, il refuse l'accès
 - si correct, il lance la procédure de login (analyse différents fichiers de configuration et met en place l'environnement de l'utilisateur)
- L'utilisateur est alors placé dans son répertoire d'accueil : c-à-d
/home/nouhaila

Changer son mot de passe

- Si vous souhaitez changer votre mot de passe, la commande pour réaliser cette opération est : **passwd** ou **yppasswd**

```
% yppasswd
```

```
Changing NIS password for USER on MACHINE
```

```
Old password:                --entrez votre mot de passe courant
```

```
New password:                --entrez votre nouveau mot de passe
```

```
Retype new password:        --rentrez votre mot de passe
```

```
NIS entry has changed on filemon
```

Quel Shell ?

- Après le login, l'utilisateur accède à un interpréteur de commandes ou shell
- Le shell affiche un «prompt» et attend les commandes de l'utilisateur
- Il en existe plusieurs avec des fonctionnalités et des interfaces différentes les uns des autres
 - sh : Bourne Shell (shell standard)
 - ksh : Korn Shell
 - csh : C Shell
 - bash : GNU (Bourne Again Shell)
- Pour savoir quel shell est utilisé, tapez :

```
% echo $SHELL  
/bin/bash
```

- La liste des shells autorisés : /etc/shells

Quelle est mon identité ?

- Pour Linux, l'identité d'un utilisateur est celle sous laquelle il se logge
- La commande **whoami** vous donne votre identité

```
% whoami  
root
```

- L'utilisateur appartient également à un ou plusieurs groupes
- La commande **id** vous donne votre identité et votre groupe

```
% id  
uid=5230(nouhaila) gid=64(profs) groups=64(profs)
```

n° de l'utilisateur

utilisateur

n° du groupe

groupe

Premières commandes : pwd et ls

- Une commande est un mot-clé avec éventuellement des options

```
% commande -options arguments
```

- La commande **pwd** (print working directory) indique le répertoire courant

```
% pwd  
/home/profs/nouhaila
```

- La commande **ls** permet d'afficher le contenu d'un répertoire

```
% ls  
Cours.tex  Examen_Linux.pdf  Recherche  Tps_Linux
```

Commande : ls avec options

- Avec l'option -l (pour version longue) plus d'informations sont affichées

```
% pwd
/home/yassine
% ls -l
-rw-r-----  1 yassine  profs   362514 Sep 5 12:40 Cours.tex
-rwxrw-r--   1 yassine  profs    1024 Sep 1  2:10 Examen_Linux.pdf
drw-r--rw-   4 yassine  profs     10 Jan 7 15:41 Recherche/
drwxrwxrwx   6 yassine  profs    8425 Mar 2 11:38 Tps_Linux/
```

- **ls** sur un fichier affiche le nom de ce fichier si celui ci existe

```
% ls Cours.tex
Cours.tex
```

Commande : ls avec options

- Les principales options sont :
 - -l : format détaillé
 - -a : liste aussi les fichiers qui commencent par « . »
 - -d : si l'argument est un répertoire, la commande liste seulement son nom et pas les fichiers qu'il contient
 - -t : affiche en triant par date de dernière modification
 - -g : affiche les informations sur le groupe

Autorisation d'accès	propriétaire	Taille du fichier	Nom du fichier
drwxrwxrwx	6 yassine	8425	Mar 2 11:38 Tps_Linux/
Type du fichier	Nb de liens	groupe	date de dernière modification

Type du fichier

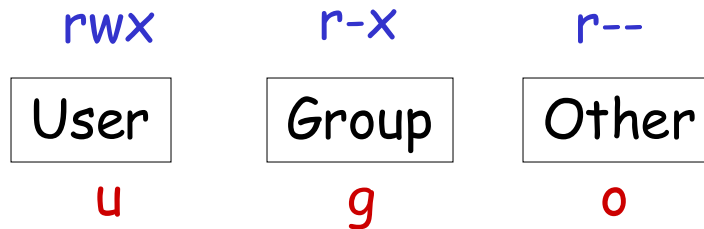
- L'indicateur du type de fichier peut prendre :
 - - : un fichier ordinaire
 - d : un répertoire
 - l : un lien symbolique
 - b : un fichier spécial de type bloc (périphériques ...)
 - c : un fichier spécial de type caractère (périphériques ...)
 - s : socket
 - ...

Droits d'accès aux fichiers

- Les fichiers possèdent un certain nombre d'attributs qui définissent les autorisations d'accès.

r autorisation à lire : read
w autorisation à écrire : write
x autorisation à l'exécution : execute

- Ces attributs sont groupés en 3 groupes de 3 attributs



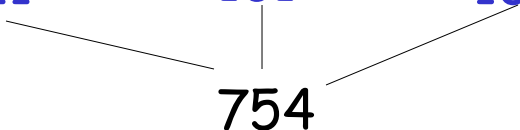
- rwxr-xr-- : fichier ordinaire : lecture, écriture et exécution permise pour le propriétaire, lecture et exécution pour le groupe et seulement lecture pour les autres. Il est donc impossible aux membres du groupe et aux autres utilisateurs d'écrire dans ce fichier

Modification des droits d'accès aux fichiers

- La protection d'un fichier ne peut être modifier que par le propriétaire
- La commande utilisée est : **chmod** (Change MODe)
- Il existe deux modes d'utilisation de cette commande :

Par un nombre octal

```
% chmod [nombre octal] fichier
```

rwX	r-X	r--
111	101	100
		

Représentation binaire

$$7 = 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$
$$5 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Symbolique

```
% chmod [who]op[permission] fichier
```

who : est une combinaison de lettre

u = user = propriétaire

g = groupe

o = other = autres

a = all = tous = ugo

op : + ajoute un droit d'accès

- supprime un droit d'accès

= affecte un droit de manière absolue

permission : r, w, x

Modification des droits d'accès aux fichiers

- Exemple :

```
% ls -l Cours.tex
```

```
-rw-r----- 1 yassine  profs  362514 Sep 5 12:40 Cours.tex
```

```
% chmod 777 Cours.tex
```

```
% ls -l Cours.tex
```

```
-rwxrwxrwx 1 yassine  profs  362514 Sep 5 12:40 Cours.tex
```

```
% chmod g-w,o-wx Cours.tex
```

```
% ls -l Cours.tex
```

```
-rwxr-xr-- 1 yassine  profs  362514 Sep 5 12:40 Cours.tex
```

```
% chmod go=r Cours.tex
```

```
% ls -l Cours.tex
```

```
-rwxr--r-- 1 yassine  profs  362514 Sep 5 12:40 Cours.tex
```


Droits d'accès à la création d'un fichier

- La protection d'un fichier, le nom du propriétaire et le nom du groupe auquel vous appartenez sont établis à sa création
- Ces paramètres ne peuvent être modifiés que par son propriétaire
- La commande permettant de définir un masque de protection des Fichiers (et répertoires) est : **umask**
- Il existe deux modes d'utilisation de cette commande :

Par un nombre octal

```
% umask [nombre base 8]
```

111	111	111	permission permanente → 754
111	101	100	

000	010	011
-----	-----	-----

023

```
% umask 023
```

Symbolique

```
% umask [who]op[permission]
```

```
% umask u=rwx,g=rx,o=r
```

```
% umask
```

```
023
```

```
% umask -S
```

```
u=rwx, g=rx, o=r
```

Droits d'accès aux répertoires

- L'interprétation des droits est différente de celle des fichiers
- Les informations concernant un répertoire est données par la commande : `ls -dl repertoire`
- L'interprétation des protections est :
 - r : autorise la lecture du contenu du répertoire, permet de voir la liste des fichiers (et sous-répertoires) contenu dans le répertoire.
 - x : autorise l'accès au répertoire (à l'aide de la commande `cd`).
 - w : autorise la création, la suppression et le changement du nom d'un élément du répertoire. Cette permission est indépendante de l'accès aux fichiers du répertoire.

Droits d'accès aux répertoires

- Exemple :

```
% ls -dl Tps_Linux/  
dr-x----- 1 yassine  profs   3625 Sep 5 12:40 Tps_Linux/  
% ls -l Tps_Linux/TP1.ps  
-rwx----- 1 yassine  profs   2514 Sep 2 10:35 TP1.ps
```

- Seul le propriétaire pourra modifier son fichier **TP1.ps**
- Mais il ne peut pas le **supprimer** car le propriétaire du répertoire **Tps_Linux** (c-à-d l'utilisateur) n'a pas l'autorisation **w** (autorisation de création, suppression, modification du nom d'un élément du répertoire)

Se déplacer dans l'arborescence

- La commande permettant de se déplacer dans une arborescence est :
`cd répertoire (change directory)`

```
% pwd  
/home/profs/yassine  
% cd Enseignement  
% pwd  
/home/profs/yassine/Enseignement
```

- Chaque répertoire contient 2 entrées supplémentaires :
 - « . » : désigne le répertoire courant
 - « .. » : désigne le répertoire parent
- On peut se déplacer en utilisant un chemin : `cd chemin`
- Deux types de chemins : `absolu` ou `relatif`

Se déplacer dans l'arborescence

- Chemin absolu : chemin qui part directement du répertoire racine

```
% pwd  
/home/profs/yassine  
% cd /home/profs/yassine/Enseignement  
% pwd  
/home/profs/yassine/Enseignement
```

- Chemin relatif : chemin qui part du répertoire courant

```
% pwd  
/home/profs/yassine  
% cd Enseignement  
% pwd  
/home/profs/yassine/Enseignement
```

Commandes liées aux répertoires

- La commande servant à créer des répertoires est :

`mkdir [options] répertoires...` (make directory)

- Il suffit d'avoir le droit d'écrire (w) dans le répertoire père

- Pour créer une arborescence entière, on utilise l'option -p
Exemple : créer l'arborescence `~/TP_Linux/TP_Groupe1`

`mkdir -p TP_Linux/TP_Groupe1`

- La commande servant à supprimer un répertoire est :

`rmdir répertoire` (remove directory)

- Le répertoire doit être vide

- Si le répertoire est non vide

`rmdir -r repertoire`

Commandes liées aux répertoires

- La commande servant à copier un fichier d'un répertoire vers un autre répertoire :

`cp fichier_source répertoire_destination (copy)`

- Pour copier des fichiers dans un répertoire :

`cp -i fichiers... répertoire_destination`

- La commande servant à copier tous les fichiers d'un répertoire :

`cp -r répertoire_source répertoire_destination`

- Toute l'arborescence du répertoire source est copiée dans le répertoire destination

- Les nouveaux fichiers se trouvent dans le répertoire :

`répertoire_destination/répertoire_source`

Les alias

- On peut lancer des commandes qui ne possèdent pas un exécutable du même nom en créant un **alias** avec la commande alias du shell

alias **nom_alias='commandes'**

```
% alias ll='ls -l'
```

```
% ll
```

```
-rw-r-----    1 yassine  profs    362514  Sep 5 12:40  Cours.tex
drw-r--rw-     4 yassine  profs           10 Jan 7 15:41  Recherche/
```

- La liste des alias peut être obtenu par la commande : **alias**

```
% alias
```

```
b='/bin'
```

```
ll='ls -l'
```

```
rm='rm -i'
```

- On peut enlever un alias en utilisant la commande : **unalias**

unalias **nom_alias**

La recherche d'un fichier : find

- La commande **find** parcourt les répertoires et leurs sous-répertoires de manière récursive à la recherche de fichiers

- La syntaxe de cette commande est :

find répertoire(s) critère_de_sélection option(s)

- Un répertoire ne peut être parcouru que si l'utilisateur dispose des droits de lecture et d'exécution sur ce répertoire ou sous-répertoire

- Options de sélection des fichiers et répertoires :

- **-print** : affiche le chemin d'accès pour chaque fichier trouvé
- **-name** : recherche par nom de fichier
- **-type** : recherche par type de fichier
- **-user** : recherche par propriétaire
- **-size** : recherche par taille du fichier
- **-atime, -amin** : recherche par date de dernier accès (jour, minute)
- **-mtime, -mmin** : recherche par date de dernière modification (jour, minute)
- **-ctime, -cmin** : recherche par date de création (jour, minute)
- **-perm** : recherche par autorisation d'accès

La recherche d'un fichier : find

```
% find . -type d -print
```

Affiche tous les répertoires contenus dans le répertoire courant

```
% find . -type f -name "*s*" -print
```

Affiche tous les fichiers contenus dans le répertoire courant et dont le nom contient la lettre s

```
% find . -type f -size +200k -print
```

Affiche tous les fichiers de plus de 200 Ko

```
% find . -mtime -3 -print
```

Affiche tous les fichiers dont la date de la dernière modification remonte à moins de trois jours

```
% find /home/TP_Linux -type d -perm 755 -print
```

Affiche tous les sous-répertoires du répertoire /TP_Linux ayant comme autorisations d'accès rwxr-xr-x

La recherche d'un mot : grep

- La commande **grep** permet de rechercher, dans un ou plusieurs fichiers, toutes les lignes qui contiennent une chaîne de caractères donnée
- La syntaxe : **grep option(s) expression fichier(s)**
- Les options :
 - **-n** : fait précéder chaque ligne affichée par son numéro de ligne dans le fichier source
 - **-v** : affiche toutes les lignes sauf celles contenant **expression**
 - **-l** : n'affiche que les noms des fichiers dont au moins une ligne satisfait à la recherche
 - **-i** : ne fait aucune distinction entre les majuscules et les minuscules
 - **-c** : affiche le nombre de lignes qui contiennent l'expression

La recherche d'un mot : grep

```
% grep read programme.c
```

Affiche toutes les lignes du fichier `programme.c` contenant `read`

```
% grep -n read programme.c
```

Affiche avec la numérotation toutes les lignes du fichier `programme.c` contenant `read`

```
% grep -i 'else do' programme.c
```

Affiche toutes les lignes du fichier `programme.c` contenant la chaîne de caractères `'else do'` en majuscules ou minuscules

```
% grep -l read *
```

Recherche tous les fichiers contenant le mot `read` et affiche leurs noms

Gestion des sorties imprimantes : lpr, lpq et lprm

- Pour demander l'impression d'un fichier (le placer dans une file d'attente), nous utilisons la commande :

`lpr -P nom_imprimante fichier`

- L'impression d'un fichier sous Linux passe par un spooler d'impression
- Le spooler est réalisé par un processus système qui s'exécute en tâche de fond

- Pour connaître l'état de la file d'attente associée à l'imprimante :

`lpq -P nom_imprimante`

- Pour retirer un fichier en attente d'impression, nous disposons de la commande :

`lprm -P nom_imprimante numéro_job`

Le `numéro_job` spécifie le numéro de job, il est obtenu grâce à la commande `lpq`

Système de fichier : file, cat

- **file nom_de_fichier** : il est possible sous Unix de connaître aussi le type de fichier sur lequel on travaille. En dehors de l'extension qui peut être trompeuse, tous les fichiers ont une en-tête permettant de déterminer leur type (répertoire, exécutable, texte ASCII, programme C, document Postscript...). L'ensemble de ces en- têtes est défini dans le fichier /etc/ magic.
- **cat [- v] [fichier...]** : affichage du contenu d'un fichier:
La commande: cat [- v] [fichier...] sert à afficher le contenu d'un fichier sice dernier est passé en paramètres. Sinon, c'est l'entrée standard qui est prise par défaut (clavier ou résultat d'une autre commande).
- **cat fichier1 fichier2 > fichier3** : cette commande permet de créer un fichier (fichier3) en concaténant les fichiers fichier1 et fichier2.

Systeme de fichier : more

more : la commande: `more [fichier...]` permet d'afficher le contenu d'un fichier page à page. Le fichier par défaut est l'entrée standard (en général le résultat de la commande située avant le «|»).

Utilisation:

- `$ ls -l | more` (redirige la sortie de la commande `ls -l` vers le `more`)
- `$ more .profile` (affiche page à page le fichier `.profile`)

En bas de chaque page de la commande `more` , un certain nombre d'action sont possibles:

- `h`: permet d'afficher la liste des commandes disponibles à ce niveau,
- `[espace]`: permet de passer à la page suivante,
- `[entrée]`: permet de passer à la ligne suivante,
- `b` : permet de revenir à la page précédente,
- `i[espace]`: permet d'afficher i lignes en plus,
- `=` : permet d'afficher le numéro de la dernière ligne affichée,
- `/mot` : permet de se positionner 2 lignes avant la prochaine occurrence du mot «mot»,
- `n`: continue la recherche précédente (identique à /),
- `:f`: permet d'afficher le nom du fichier et le numéro de dernière ligne affichée,
- `.` : effectue de nouveau la commande précédente,

Création et consultation d'un fichier

- Création de fichier: `$ touch fichier`

cette commande permet de créer un fichier vide portant le nom fichier

- Commandes de base pour consulter le contenu d'un fichier de texte :

- `cat fich`, `more fich` : affichage simple et page par page ;
- `head fich`, `head -n fich` : affichage des n premières lignes ;
- `tail fich`, `tail -n fich` ; affichage des n dernières lignes ;
- `wc [-option] fich` : affichage du nombre de lignes, de mots, de caractères

- Options:

- `-l` : pour le nombre de lignes
- `-w` : pour le nombre de mots
- `-c` : pour le nombre de caractères.

Commandes Liées aux répertoires : mv

■ Le renommage et le déplacement d'une entrée de l'arborescence s'effectuent avec la commande mv (move).

\$ mv R1/exemple R2/nouveau

■ Le renommage consiste simplement à modifier le nom de l'entrée dans le contenu du répertoire.

- Le déplacement consiste à supprimer l'entrée du répertoire d'origine et à insérer
- une nouvelle entrée dans le répertoire destination.
- Les blocs de données ne sont en aucun cas déplacés.

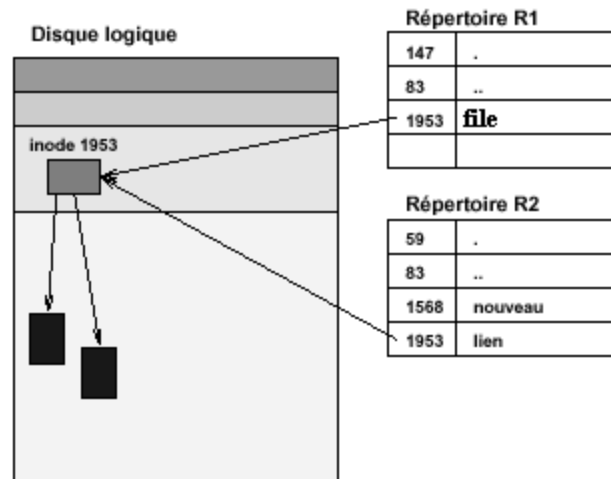
Concept du lien physique

- Un lien dit **physique** est une relation entre un répertoire et un **fichier** : plusieurs liens sur une entrée signifie donc plusieurs repérages dans l'arborescence du même inode et du même contenu.
- Il est possible **de créer un nouveau lien** sur une entrée déjà existante avec la commande `ln (link)`.

`$ ln file ../R2/lien`

Une nouvelle entrée dans le répertoire R2 est créée.

- L'inode 1953 est accessible à partir de R1 et de R2 respectivement par les noms `file` et `lien`.

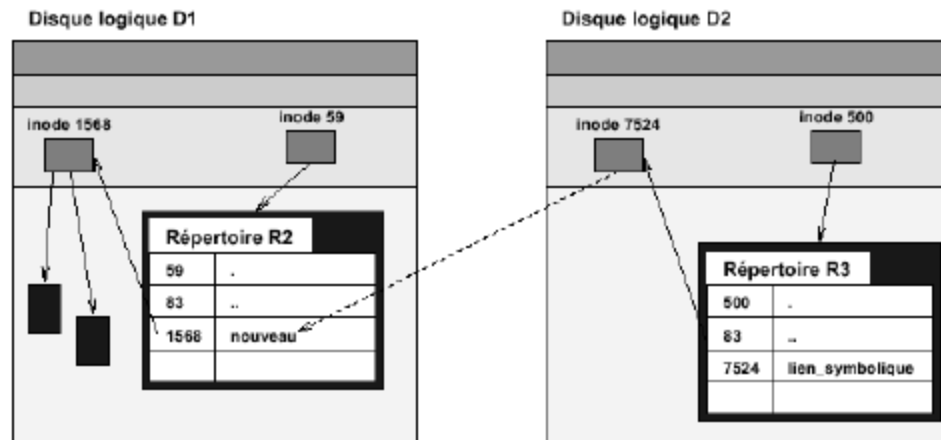


Concept du lien symbolique

- Un lien physique utilisant un numéro d'inode, il se limite à la même partition.
- Pour traverser les limites des partitions, Unix introduit des liens dits symboliques.
- La création d'un lien symbolique s'effectue avec l'option -s de la commande ln.

```
$ ln -s ../R2/nouveau ../R3/lien_symbolique
```

- Un nouvel inode est créé dans la partition d'origine.
- Cet inode contient le nom (absolu ou relatif) de l'élément pointé, au lieu d'un numéro d'inode.



Les éditeurs de texte

L'éditeur vi

- vi est un éditeur entièrement en mode texte : chacune des commandes se fait à l'aide de commandes texte
- vi est peu pratique, très puissant, très utile en cas de non fonctionnement de l'interface graphique
- Quand vi devient actif :
 - un ~ apparaît à gauche de chaque ligne de l'écran,
 - vi est alors en **mode commande** et attend votre première instruction
- vi possède 2 modes :
 - **mode commande** : permet de taper des commandes
 - **mode insertion** : permet de saisir du texte en ajoutant du texte après ou avant le curseur
- Pour passer du **mode commande** en **mode insertion**, tapez :
 - **a** pour insérer du texte **après** le curseur
 - **i** pour insérer du texte **avant** le curseur

Créer un fichier vi

- Lancer vi en tapant `vi`
- Un écran comportant une colonne remplie de tildes s'affiche
- Passer du `mode commande` en `mode insertion` en appuyant sur la touche `a` (n'appuyer pas sur Entrée)
- Vous pouvez insérer des caractères sur la première ligne. Le caractère `a` n'apparaîtra pas à l'écran
- Ajouter des lignes de texte, vous pouvez utiliser la touche `Correction` pour supprimer les erreurs de la ligne en cours
- Passer du mode insertion en mode commande en appuyant sur la touche `Echap`
- Enregistrer en tapant : `:w nom_du_fichier`
- La ligne d'état confirme cet action en affichant :
 `"nom_du_fichier" [New File] 4 lines, 46 characters`
- Quitter vi en tapant : `:q`

Quelques commandes : vi

- Commandes de base :

:q	quitte l'éditeur
:q!	force l'éditeur à quitter
:wq	sauvegarde le document et quitte l'éditeur
:nom_du_fichier	sauvegarde le document sous le nom nom_du_fichier

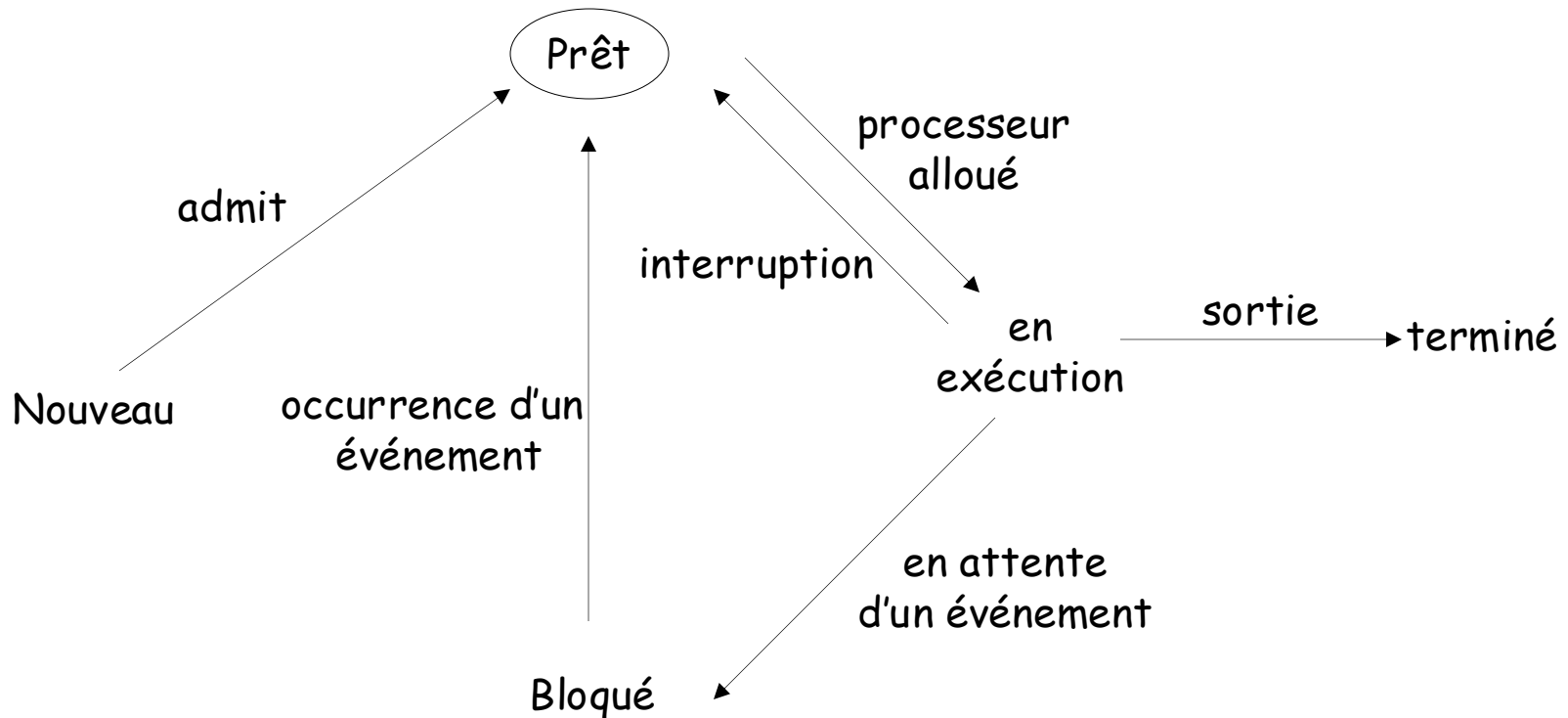
- Commandes d'édition :

x	efface le caractère actuellement sous le curseur
dd	efface la ligne actuellement sous le curseur
dxd	efface x lignes à partir de celle actuellement sous le curseur
nx	efface n caractères à partir du caractère actuellement sous le curseur
r	remplace le caractère actuellement sous le curseur
cw	modifie le mot courant à partir de la position du curseur
cc	modifie la ligne entière

Concept de processus

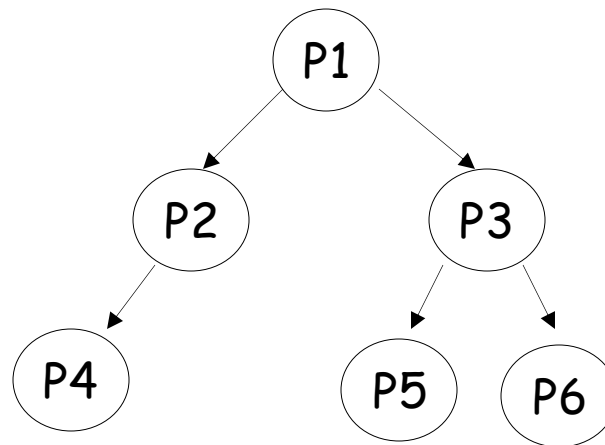
Concept de processus

- Un processus est un programme en cours d'exécution
- Le processeur traite une tâche à la fois, s'interrompt et passe à la suivante
- Le diagramme d'état du processus



Création d'un processus

- Les processus des utilisateurs sont lancés par un interprète de commande (shell). Ils peuvent eux même lancer ensuite d'autres processus
- Ces processus doivent ensuite pouvoir communiquer entre eux
- Le processus créateur = le père
- Les processus créés = les fils
- Les processus peuvent se structurer sous la forme d'une arborescence



Destruction d'un processus

- 3 possibilités pour l'arrêt d'un processus
 - Normal : par lui même en ayant terminé ses opérations
 - Autorisé : par son père qui exécute une commande appropriée
 - Anormal : par le système
 - temps d'exécution dépassé
 - mémoire demandée non disponible
 - instruction invalide
 - etc.
- Le processus créateur est le seul à pouvoir exécuter l'arrêt de ses fils
- Dans plusieurs systèmes, la destruction d'un processus père entraîne la destruction de tous ses fils

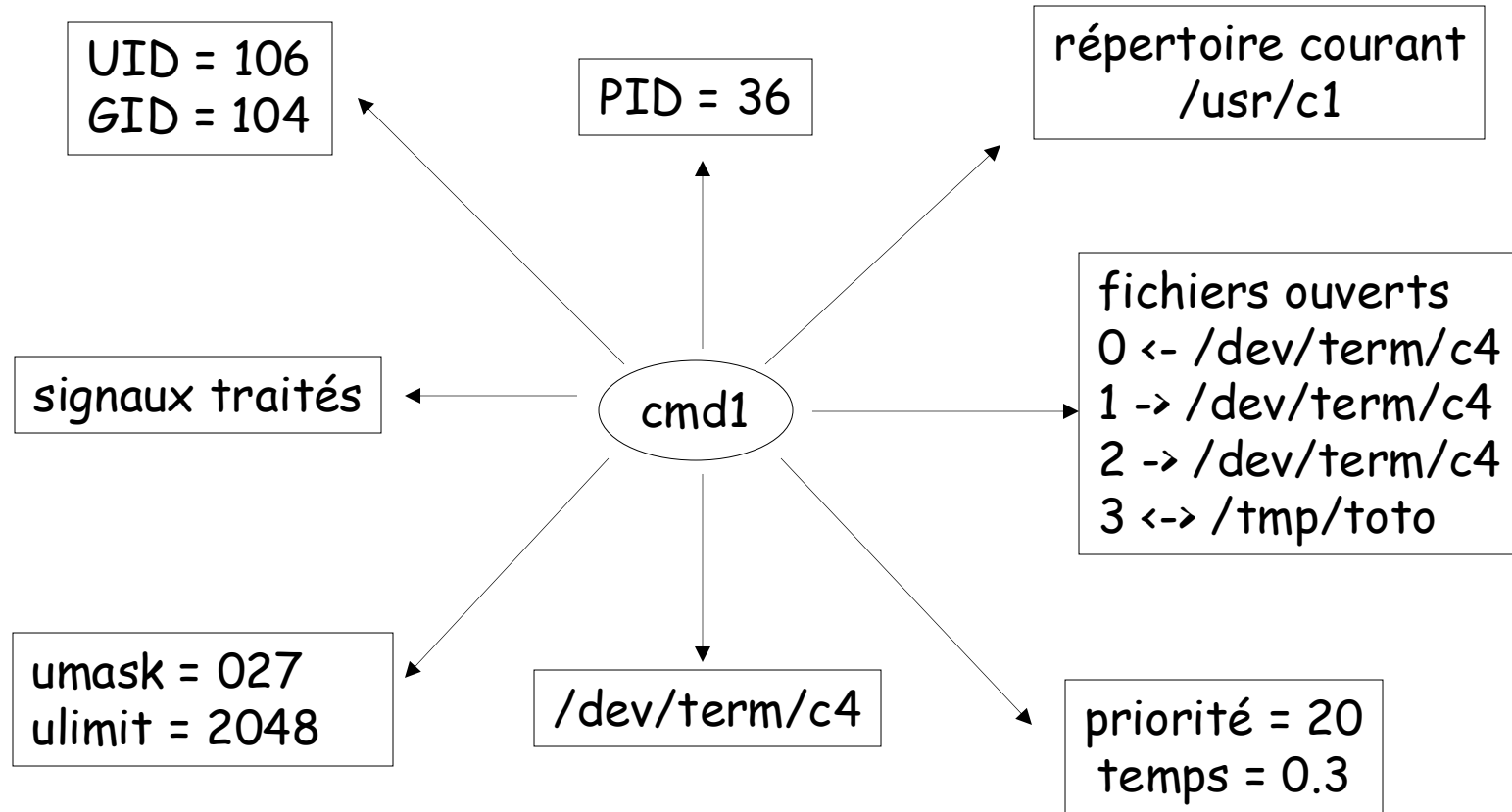
Mise en oeuvre

- Pour mettre en oeuvre le modèle des processus, le système d'exploitation construit une table, appelé table des processus, dont chaque entrée correspond à un processus particulier
- Chaque entrée comporte des informations sur :
 - l'état du processus
 - son compteur ordinal : contient l'adresse de la prochaine instruction à extraire de la mémoire
 - son pointeur de pile : contient l'adresse courante du sommet de pile en mémoire
 - son allocation mémoire
 - l'état de ses fichiers ouverts
 - et tous ce qui peut être sauvegardé lorsqu'un processus passe de l'état élu à l'état prêt

Structure d'un processus

- L'environnement d'un processus comprend :
 - un numéro d'identification unique appelé **PID** (Process IDentifier)
 - le numéro d'identification de l'utilisateur qui a lancé ce processus, appelé **UID** (User IDentifier), et le numéro du groupe auquel appartient cet utilisateur, appelé **GID** (Group IDentifier)
 - le répertoire courant
 - les fichiers ouverts par ce processus
 - le masque de création de fichier, appelé **umask**
 - la taille maximale des fichiers que ce processus peut créer, appelé **ulimit**
 - la priorité
 - les temps d'exécution
 - le terminal de contrôle, c'est à dire le terminal à partir duquel la commande a été lancée, appelé **TTY**

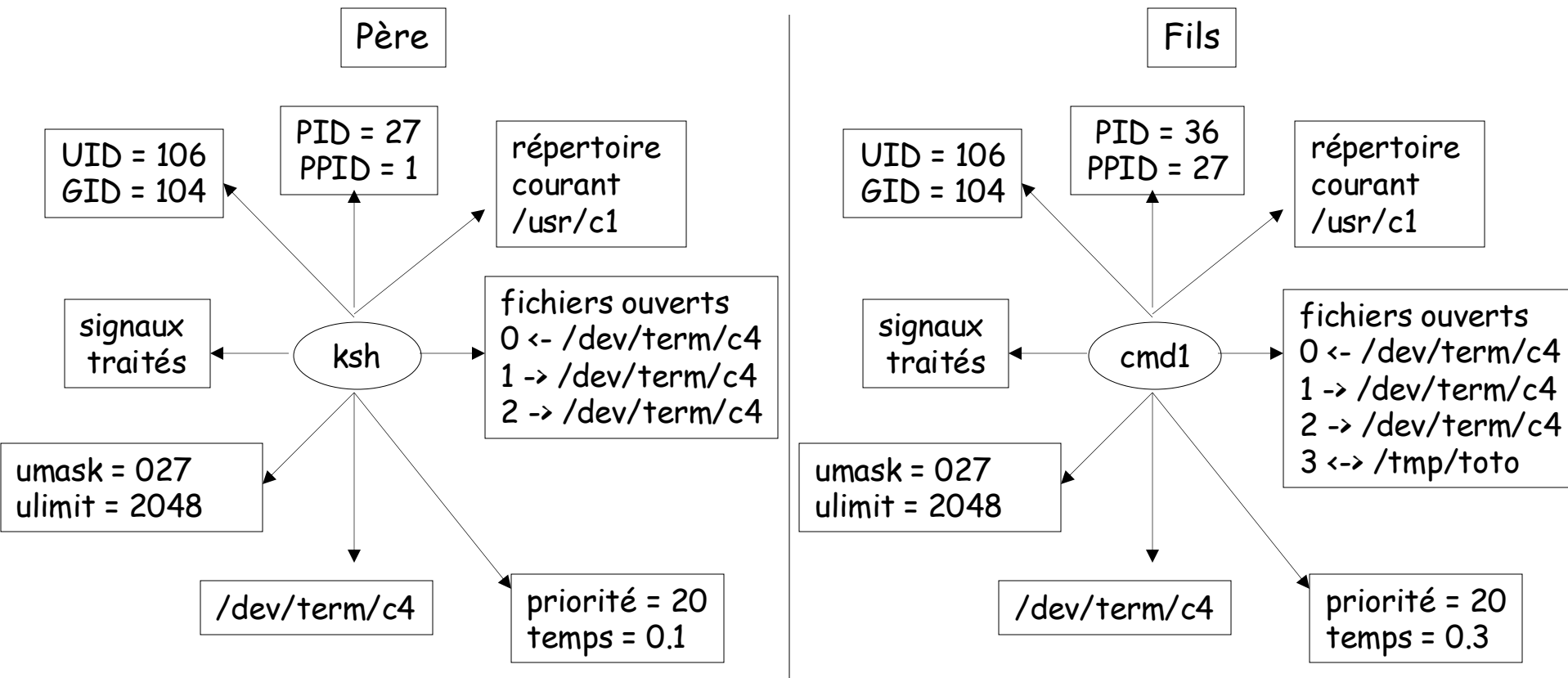
Un exemple : schéma d'un processus Unix



Ce processus a le numéro 36. Il a été lancé par l'utilisateur qui a 106 pour UID. Il est entrain d'exécuter le programme 'cmd1'. Il a consommé 0.3 seconde, avec une priorité de 20. Son masque de création est 027. Son terminal de contrôle est /dev/term/c4. Son répertoire courant est /usr/c1. Il a 4 fichiers ouverts : 0, 1, 2, et 3.

Structure d'un processus Unix

- Le **PPID** est le **PID** du processus père
- Le processus fils hérite de tout l'environnement du processus père, sauf bien sûr du PID, du PPID et des temps d'exécution
- Le père du processus 36 est le processus 27, et celui de 27 est le processus 1
- Seul le fils 36 a ouvert le fichier /tmp/toto



Les processus : la commande ps

- Un processus est un programme qui est en cours d'exécution
- La commande **ps** donne un ensemble de renseignements sur les processus en cours d'exécution
- Syntaxe : **ps options**
- Options :
 - **-a** : affiche des renseignements sur tous les processus attachés à un terminal
 - **-l** : donne, pour chaque processus, le nom de l'utilisateur (user), le pourcentage de cpu (%cpu), la taille totale du processus dans la mémoire (size), la mémoire réservée (rss) en Ko ...
 - **-x** : affiche également des informations sur les processus non liés au terminal
 - **-w** : affiche sur 132 colonnes, utile pour voir le nom complet de la commande associée à chaque processus

Les processus : la commande ps

% ps				
PID	TTY	STAT	TIME	CMD
746	pts/3	S	00:00:00	-bash
749	pts/3	S	00:00:02	gs
848	pts/3	S	00:03:28	mozilla-bin
965	pts/3	S	00:00:00	ps

- PID : le numéro d'identification du processus
- TTY : le terminal depuis lequel le processus a été lancé
- STAT : l'état du processus au moment du lancement de la commande
 - R : le processus est en cours d'exécution
 - T : le processus est stoppé
 - S : le processus dort depuis moins de 20 secondes
 - Z : le processus en attente d'un message du noyau
- TIME : le temps d'exécution de la commande
- CMD : le libellé de la commande lancée

Arrêt d'un processus : kill

- La commande kill permet d'envoyer un signal au processus

- Syntaxes :

```
kill -signal pid  
kill -l
```

- Options :

- **-9** : demande l'arrêt du processus désigné par son **pid**
- **-l** : affiche la liste des signaux disponibles

```
% kill -l
```

```
1) HUP 2) INT 3) QUIT ...
```

```
7) EMT 8) FPE 9) KILL ...
```

```
% kill -9 1635
```

Cette commande tue le processus dont le numéro **PID** est **1635**

Prof : Rachida AIT ABDEOUAHID